

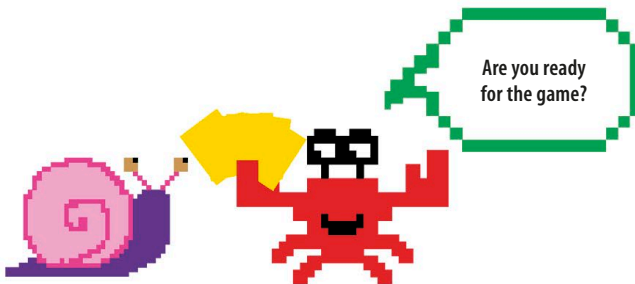
10 Shuffle the shapes

To ensure that the shapes don't appear in the same order each time, you need to shuffle them – just like you would do with a pack of cards. The `shuffle()` function in `random` can do this for you. Insert this line after `c.pack()`.

```
random.shuffle(shapes)
```

Getting ready

In the next part of the build, you'll set up several variables and write a few bits of code that get the game ready for playing. However, it won't work until we add the functions in the last stage.



11 Set up variables

You'll need variables to keep track of various things while the program is running, including the current shape, the previous and current colour, and the two players' scores.

Neither player has any points at the start, so the value of both is set to 0.

```
random.shuffle(shapes)
```

```
shape = None
```

```
previous_color = ''
```

```
current_color = ''
```

```
player1_score = 0
```

```
player2_score = 0
```

The `shape` variable has no value yet.

The `color` variables hold an empty string.

12 Add a delay

Now add a line to create a 3-second delay before the first shape appears. This gives the player time to find the `Tkinter` window in case it's hidden behind other windows on the desktop. You'll create the `next_shape()` function later, in Steps 16 and 17.

```
player2_score = 0
```

```
root.after(3000, next_shape)
```

The program waits for 3,000 milliseconds, or 3 seconds before showing the next shape.

EXPERT TIPS

Nothing really matters

Coders often need to set up variables with a starting value of zero, such as the scores in this game. But how do you do this if a variable holds a string rather than a number? The answer is to use a pair of quote marks with nothing between them. Some variables, however, don't have an obvious default value such as 0 or an empty string. In that case, you can use the word "None", as we do below.